MyCodeSchool

Testare cu JUnit și Mockito — exerciții

Alex · exercițiile 1–4: JUnit, primul mock, verify, @InjectMocks.

Proiectul e magazinul online din `src/`. Testele le scrii în `test/`, în același pachet ca clasa testată. Modelul e `test/app/system/cart/CartTest.java`: îl citești întâi, apoi rulezi `mvn test` și vezi `Tests run: 1, Failures: 0`.

Fiecare test: `// Arrange`, `// Act`, `// Assert`, fără bucle, numele spune ce se întâmplă și când. După fiecare exercițiu, `mvn test` e verde.

Exercițiul 1 · Primul test, fără mock

Clasa `ProductTest` în `test/app/products/model/`. `Product` are un constructor care citește o linie din `Products.txt`: `id,nume,pret,greutate,categorie,stoc`.

1. `constructorReadsAllFieldsFromLine`: din `"1,Laptop,3000,2000,IT,5"` ies id `1`, nume `Laptop`, preț `3000`, greutate `2000`, categorie `IT`, stoc `5`. Câte un `assertEquals` pe fiecare câmp.
2. `toStringReturnsTheSameLine`: `toString()` pe produsul de mai sus dă exact linia din care a fost creat.
3. `constructorThrowsWhenPriceIsNotANumber`: linia `"1,Laptop,trei mii,2000,IT,5"` aruncă `NumberFormatException`.
4. `constructorThrowsWhenFieldsAreMissing`: linia `"1,Laptop"` aruncă tot o excepție. Află care: rulează o dată cu `assertThrows(Exception.class, ...)`, apoi pune tipul exact.

Done când: `Tests run: 5, Failures: 0`.

Exercițiul 2 · Primul mock: totalul coșului

În `CartTest`, `totalPriceMultipliesPriceByQuantityForEachItem`. Coșul are două produse:

Produs	id	preț	cantitate
laptop	1	3000	1
mouse	2	50	3

Produsele (`Product`) și rândurile din coș (`CartItem`) sunt `@Mock` , niciunul creat cu `new` . Le dai valorile cu `when(...).thenReturn(...)` , le pui în coș cu `addToCart` , apoi `totalPrice()` trebuie să fie `3150` .

Vezi în `Cart` ce metode cheamă `addToCart` și `totalPrice()` pe un `CartItem` și pe un `Product` : doar pe acelea pui `when` . Un `when` în plus face testul să pice cu `UnnecessaryStubbingException` .

Done când: `Tests run: 6, Failures: 0` .

Exercițiul 3 · verify: același produs de două ori

Tot în `CartTest` :

1. `addToCartIncrementsQuantityWhenProductIsAlreadyInCart` : două rânduri diferite (`laptopItem` , `secondLaptopItem`), dar cu același produs (id `1`). Primul are cantitatea `2` . După ce le adaugi pe amândouă, coșul are tot un singur rând, iar pe primul s-a apelat `setQuantity(3)` .
2. `removeFromCartLeavesCartEmptyWhenItWasTheOnlyItem` : adaugi un rând, îl scoți, coșul e gol.

`laptopItem` e un mock, deci `getQuantity()` întoarce mereu `2` , oricâte `setQuantity` primește. De aceea nu verifici cantitatea cu `assertEquals` , ci cu `verify` : ce i s-a cerut rândului.

Done când: `Tests run: 8, Failures: 0` .

Exercițiul 4 · @InjectMocks pe un serviciu

Clasa `ProductCommandServiceImplTest` în `test/app/products/service/` , cu `ProductRepository` și `ProductMapper` ca `@Mock` și serviciul `@InjectMocks` . Produsele cu care lucrezi sunt și ele `@Mock` .

1. `addProductSavesProductWhenNameIsNew` : repository-ul nu găsește niciun produs cu numele `Laptop` . După `addProduct` , repository-ul a primit `addProduct` cu exact acel produs.
2. `addProductThrowsWhenNameAlreadyExists` : repository-ul găsește deja un alt produs cu numele `Laptop` . `addProduct` aruncă `ProductAlreadyExistsException` , iar repository-ul nu primește niciun `addProduct` .

3. `removeProductThrowsWhenProductDoesNotExist`: produsul are id-ul `7`, iar repository-ul nu găsește nimic cu id-ul ăsta. `removeProduct` aruncă `ProductNotFoundException`, iar repository-ul nu primește niciun `removeProduct`.

`addProduct` aruncă o excepție verificată, deci testul de la punctul 1 are `throws Exception` în semnătură.

Verifică testele: în `ProductCommandServiceImpl.addProduct`, mută `productRepository.addProduct(product);` deasupra lui `if`. Testul 2 trebuie să pice cu `Never wanted here`. Pune linia înapoi.

Done când: `Tests run: 11, Failures: 0`.